

WestQuant Representation Stack (Combined)

Slide 1 — Project Overview

Problem: Quantum algorithm design typically performs one fixed translation from optimization problem to hardware. We argue representation itself should be searched.

Approach: A composable pipeline: $P \rightarrow H_i$ (Hamiltonian Explorer) $\rightarrow R_{ij}$ (Representation Scheduler) $\rightarrow Q_{ijk}$ (QoolQit compilation + emulation) $\rightarrow$ Pareto selection. Both projects are independently useful and composable.

Results: Factorial experiment (3 Hamiltonians x 3 embeddings = 9 cells) on MWIS: Hamiltonian choice drives 62% of variance, embedding 10%. The best (H, R) combination outperforms the worst by $>2\times$ in solution probability.

Importance: Representation can be treated as an optimization variable. QoolQit serves as the evaluation engine; WestQuant AI is optional. Fully open-source, deterministic, reproducible.

WestQuant Representation Stack (Combined)

Slide 2 — QoolQit Experience

Positive: QoolQit provides a clean end-to-end stack: Register, Drive, QuantumProgram, compile_to(device), LocalEmulator. The embedding API (InteractionEmbedder, SpringLayoutEmbedder, Blade) covers the main approaches. DataGraph bridges logical and physical.

Challenges: DMM requires AnalogDeviceWithDMM (not AnalogDevice). max_energy profile needed for auto-rescaling. Bitstring results are Counters of strings. Native Rydberg $J > 0$ constrains which Hamiltonians are directly realizable.

Workflow: Successive halving (cheap metrics -> emulation) is essential for search scalability. The five-class equivalence taxonomy ensures scientific rigor. Variance decomposition quantifies how much each representation layer matters.